로봇 SW 기초 교육 · 1차 퀴즈

이름 : _____

5일차 평가 · Python / Git·GitHub / OpenCV·YOLO · 100점 만점 · 90분

시험 규칙

- **AI 도구(ChatGPT, Copilot, Claude, Gemini 등) 사용 금지.** VS Code 의 AI 자동완성도 끕니다. 적발 시 0점 처리됩니다.
- 교육자료 열람 여부는 강사 안내에 따릅니다. 옆 사람과 상의할 수 없습니다.
- 채점은 프로그램이 자동으로 합니다. **파일 이름, 함수·클래스 이름, 변수 이름을 바꾸면 0점**이 됩니다.
- 제출은 GitHub 에 push 한 저장소로 합니다. push 는 여러 번 해도 되니, **종료 10분 전에는 반드시 한 번 push** 해 두세요.

구분	내용	배점
Part A · Python	함수 3문제 + 클래스·예외처리 1문제 (`python_quiz.py`)	50
Part B · Git / GitHub	저장소 만들기부터 push 까지 (STEP 1, 3, 4, 5 의 Git 작업)	30
Part C · 객관식	OpenCV 5문제 + YOLO 5문제 (`answers.py`)	20

아래 **STEP 1 → 5 순서대로** 진행하면 세 파트가 모두 끝납니다. 받은 `quiz1` 폴더를 홈 폴더에 두고(`~/quiz1`) 그 안에서 작업합니다.

받은 파일	설명
`python_quiz.py`	Part A 답을 작성하는 파일
`answers.py`	Part C 객관식 답을 적는 파일
`memo.txt` , `debug/`	GitHub 에 **올리면 안 되는** 파일과 폴더

STEP 1 · Git 저장소 준비 Part B

1. `~/quiz1` 폴더를 Git 저장소로 만듭니다.
2. `README.md` 파일을 만들고 첫 줄에 `# 본인이름` 을 적습니다. (예: `# 홍길동`)
3. `memo.txt` 파일과 `debug` 폴더가 Git 에 기록되지 않도록 설정합니다.
4. 나머지 파일을 모두 담아 첫 커밋을 합니다. 커밋 메시지 : `docs: 시험 시작`

> **커밋 메시지 규칙** — 이번 시험의 모든 커밋 메시지는 `feat:` `fix:` `docs:` `chore:` 중 하나로 시작해야 합니다. (예: `feat: 문제 1 풀이`) 시험이 끝났을 때 커밋이 **5개 이상**이어야 합니다.

STEP 2 · Python 문제 풀기 Part A · 50점

`python_quiz.py` 의 `pass` 를 지우고 코드를 작성합니다. 결과는 `print` 가 아니라 반드시 `return` 으로 돌려줍니다. **문제 하나를 풀 때마다 커밋을 하나씩** 남기세요. `python3 python_quiz.py` 로 실행하면 예시 결과와 비교할 수 있습니다.

문제 1. 배달비 계산 8점

`delivery_fee(total, distance)` — 주문 금액 `total`(원)과 배달 거리 `distance`(km)를 받아 배달비를 return 합니다.

조건	배달비
주문 금액이 30000원 이상 (거리와 상관없음)	0
그 외, 거리가 2km 미만	2000
그 외, 거리가 2km 이상 5km 미만	3000
그 외, 거리가 5km 이상	5000

```
delivery_fee(12000, 3)    # 3000
delivery_fee(35000, 8)    # 0
delivery_fee(9000, 1.5)   # 2000
```

문제 2. 운동장 이벤트 당첨자 찾기 12점

`find_winners(numbers)` — `numbers` 는 학생들이 좋아하는 숫자가 번호 순서대로 들어 있는 리스트입니다. (첫 번째 값이 **1번** 학생의 숫자) 좋아하는 숫자가 **10 이상 17 이하**이면 당첨입니다. 당첨된 학생의 **번호**를 리스트로 return 합니다. 당첨자가 없으면 빈 리스트를 return 합니다.

```
find_winners([10, 20, 17, 9])   # [1, 3]   → 1번(10), 3번(17) 당첨
find_winners([1, 2, 3])         # []
```

문제 3. 주문 금액 계산 12점

`order_total(menu, orders)` — `menu` 는 `메뉴 이름: 가격` 딕셔너리, `orders` 는 손님이 주문한 메뉴 이름 리스트입니다. (같은 메뉴가 여러 번 나올 수 있음)

- 메뉴판에 있는 메뉴는 가격을 총 금액에 더합니다.
- 메뉴판에 없는 메뉴는 금액에 더하지 않고, 몇 개인지만 셉니다.
- **총 금액**과 **메뉴판에 없는 메뉴의 개수**, 두 값을 이 순서대로 return 합니다.

```
menu = {"비빔우동": 7000, "튀김우동": 8000, "유부초밥": 4000}
order_total(menu, ["비빔우동", "유부초밥", "비빔우동", "돈까스"])   # (18000, 1)
order_total(menu, [])                                              # (0, 0)
```

문제 4. 비빔우동 키오스크 2.0 18점

대박 난 비빔우동 집의 키오스크를 클래스로 만듭니다. 비빔우동은 한 그릇에 7000원입니다. (`PRICE` 로 이미 들어 있음)

1. 매진을 뜻하는 에러 클래스 `SoldOutError` 를 직접 만듭니다.
2. `UdonKiosk(stock)` 으로 키오스크를 만들면 `self.stock` 에 오늘 판매할 개수 `stock` 이, `self.sold` 에 0 이 저장됩니다. (`self.stock` : 남은 개수, `self.sold` : 지금까지 판 개수)
3. `order(text)` — `text` 는 손님이 키오스크에 입력한 **문자열**입니다. 아래 순서대로 확인합니다.

순서	상황	동작
①	남은 개수가 0 (매진)	`SoldOutError` 를 발생시킨다
②	정수가 아니거나(`"abc"` , `"1.5"`) 1 미만 (`"0"` , `"-3"`)	`ValueError` 를 발생시킨다
③	남은 개수보다 많이 주문	`ValueError` 를 발생시킨다
④	정상 주문	남은 개수를 줄이고, 판 개수를 늘리고, **결제 금액(개수 × 7000)**을 return 한다

   에러가 난 주문은 `self.stock` , `self.sold` 를 바꾸지 않습니다.
4. `total_sales()` — 지금까지의 총 매출(판 개수 × 7000)을 return 합니다.

```
kiosk = UdonKiosk(3)
kiosk.order("2")        # 14000        (stock 1, sold 2)
kiosk.order("abc")      # ValueError
kiosk.order("5")        # ValueError   (1그릇밖에 안 남음)
kiosk.order("1")        # 7000         (stock 0, sold 3)
kiosk.order("1")        # SoldOutError (매진)
kiosk.total_sales()     # 21000
```

STEP 3 · 객관식 + branch

Part C · 20점 / Part B

1. `feature/answers` 라는 이름의 branch 를 만들고 그 branch 로 이동합니다.
2. 아래 객관식 10문제의 답을 `answers.py` 에 숫자로 적고 커밋합니다. (예: `Q1 = 3`)
3. `main` branch 로 돌아와 `feature/answers` 를 `main` 에 합칩니다. **branch 는 지우지 않습니다.**

Q1. [OpenCV] 아래 코드에서 `cv2.waitKey(0)` 의 역할로 알맞은 것은?

```
cv2.imshow("My_Love", img)
cv2.waitKey(0)
cv2.destroyWindow("My_Love")
```

① 0ms 만 기다린 뒤 창을 바로 닫는다

② 아무 키나 입력될 때까지 기다린다

③ 이미지를 파일로 저장한다

④ 창의 크기를 0 으로 만든다

Q2. [OpenCV] `cv2.imread` 로 읽은 이미지를 `plt.imshow(img)` 로 그대로 띄웠더니 원본과 색이 다르게 보였다. 이유와 해결 방법으로 알맞은 것은?

① OpenCV 는 색을 BGR 순서로 다루기 때문 → `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` 로 변환한다

② 이미지가 회색으로 읽혔기 때문 → `cv2.IMREAD_GRAYSCALE` 옵션을 준다

③ matplotlib 이 이미지를 뒤집기 때문 → `cv2.flip` 으로 다시 뒤집는다

④ 이미지 크기가 달라졌기 때문 → `cv2.resize` 로 크기를 맞춘다

Q3. [OpenCV] `print(img.shape)` 의 결과가 `(631, 630, 3)` 이었다. 세 숫자의 의미를 순서대로 바르게 쓴 것은?

① (가로, 세로, 채널)

② (채널, 행, 열)

③ (행, 열, 채널)

④ (열, 행, 밝기)

Q4. [OpenCV] 이미지를 **좌우**로 반전시키는 코드는?

① `cv2.flip(img, 0)`

② `cv2.flip(img, -1)`

③ `cv2.resize(img, (-1, 1))`

④ `cv2.flip(img, 1)`

Q5. [OpenCV] `cap.get(cv2.CAP_PROP_FPS)` 값이 24 인 영상을 원래 속도에 가깝게 재생하려고 한다. 한 프레임마다 `cv2.waitKey()` 에 넣어 줄 지연시간(ms)으로 가장 알맞은 것은?

① 약 10

② 약 24

③ 약 41.67

④ 약 1000

Q6. [YOLO] `model = YOLO("yolov8n.pt")` 에 대한 설명으로 알맞은 것은?

① 구조만 정의된, 아무것도 학습되지 않은 새 모델을 만든다

② COCO 데이터로 이미 학습된 사전학습 모델을 불러온다

③ 학습한 모델을 ONNX 형식으로 변환한다

④ coco8 데이터셋을 다운로드한다

Q7. [YOLO] `model.train(data=data_path, epochs=30, imgsz=320)` 에서 `epochs=30` 의 의미는?

① 이미지를 30장만 사용한다

② 30분 동안 학습한다

③ 클래스가 30개이다

④ 전체 데이터셋을 30번 반복해서 학습한다

Q8. [YOLO] 학습한 모델로 웹캠 영상을 확인했더니 사과가 아닌 물체까지 사과로 검출되었다. 가장 먼저 해 볼 조치는?

① `conf` 값을 올린다 (예: 0.7 → 0.8)

② `conf` 값을 내린다 (예: 0.7 → 0.3)

③ `epochs` 를 1 로 줄여 다시 학습한다

④ `VideoCapture(0)` 의 숫자를 바꾼다

Q9. [YOLO] 학습이 끝나면 `weights` 폴더에 `best.pt` 와 `last.pt` 가 저장된다. `best.pt` 는 무엇인가?

① 마지막 에폭의 모델

② 학습하기 전의 원본 모델

③ 검증 성능이 가장 좋았던 모델

④ ONNX 형식으로 변환된 모델

Q10. [YOLO] 사과(red, green) 데이터셋의 `data.yaml` 에 대한 설명으로 **틀린** 것은?

```
names:
- red
- green
nc: 2
train: /content/drive/MyDrive/apple/train/images
val: /content/drive/MyDrive/apple/val/images
```

① `nc` 는 클래스 개수이다

② `names` 의 순서는 `classes.txt` 에 적힌 순서와 같아야 한다

③ `train` , `val` 에는 각 images 폴더의 경로를 적는다

④ `names` 의 순서는 바뀌어도 학습 결과에 영향이 없다

STEP 4 · 실수 되돌리기 Part B

1. `main` 에서 `mistake.txt` 파일(내용은 아무거나)을 만들고 커밋합니다. 커밋 메시지 : `chore: 실수 커밋`
2. 방금 커밋을 **기록을 지우지 않는 방법**으로 취소합니다. (되돌리는 새 커밋이 하나 더 생기는 방법)
3. 확인 : `mistake.txt` 는 폴더에서 사라지고, `git log` 에는 두 커밋이 모두 남아 있어야 합니다.

STEP 5 · GitHub 에 제출 Part B

1. GitHub 에 `robot-quiz1` 이라는 이름의 **Public** 빈 저장소를 만듭니다. (README 추가는 Off)
2. 내 PC 저장소에 원격 저장소를 등록하고 `main` 을 push 합니다.
3. `feature/answers` branch 도 GitHub 에 push 합니다.
4. GitHub 웹에서 확인한 뒤, **저장소 주소를 강사에게 제출**합니다.

> **제출 전 확인**
>
> - GitHub 저장소에 `python_quiz.py` , `answers.py` , `README.md` 가 보인다.
> - `memo.txt` 와 `debug` 폴더는 보이지 않는다.
> - branch 가 `main` , `feature/answers` 두 개 보인다.
> - `python3 python_quiz.py` 를 실행했을 때 에러가 나지 않는다. (문법 오류가 있으면 Part A 전체가 0점)
> - push 한 뒤에 파일을 고쳤다면 다시 커밋하고 push 한다.